Jobsheet Praktikum: Pertemuan 6

Inheritance

Mata Kuliah	Praktikum Pemrograman Berbasis Objek (RTI253008)
Pertemuan	6 (Minggu 6)
Durasi	1 × 4 × 50’ praktikum; 1 × 1 × 50’ tugas/laporan mandiri

A. Capaian Praktikum

Setelah menyelesaikan jobsheet ini, mahasiswa mampu:

1. Menulis subclass yang mewarisi atribut dan method dari sebuah superclass memakai `extends` dan `super(...)`.
2. Menambahkan atribut dan method baru pada subclass tanpa mengubah superclass-nya.

B. Persiapan dan Prasyarat

- **Alat**: JDK 17 atau lebih baru, NetBeans (editor yang digunakan sepanjang praktikum ini).
- **Proyek**: pertemuan ini melanjutkan proyek `bank-mini` dari Pertemuan 4.

> **Tanpa NetBeans?** Jobsheet ini tetap dapat diikuti menggunakan editor teks biasa:
>
> ```
> javac -d out src/id/ac/polinema/*.java
> java -cp out id.ac.polinema.Main
> ```
>
> Checkpoint dan output yang dihasilkan tetap sama persis, apa pun editor yang digunakan.

C. Langkah Kerja

Langkah 1: SavingsAccount, Subclass Pertama

Proyek `bank-mini` dari Pertemuan 4 dilanjutkan pada pertemuan ini. Sejauh ini hanya ada satu jenis rekening, `Account`. Bank Mini yang sesungguhnya melayani beberapa jenis rekening dengan sifat masing-masing, tanpa menulis ulang `accountNumber`, `owner`, `balance`, `deposit()`, atau `withdraw()` dari nol. Tambahkan kelas `SavingsAccount` yang mewarisi seluruhnya dari `Account` lewat `extends`, ditambah atribut dan method barunya sendiri:

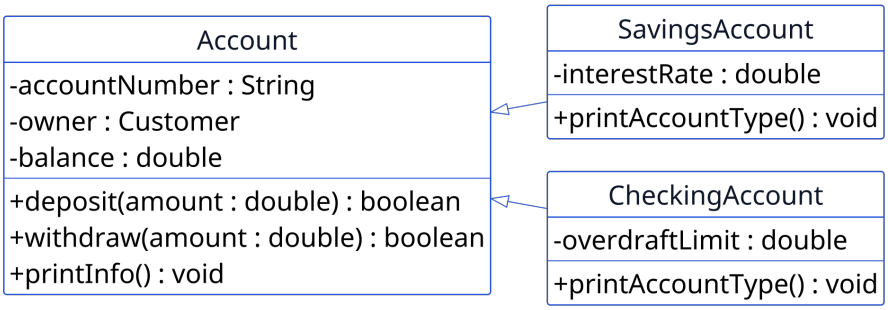


Figure 1: Diagram kelas Account, SavingsAccount, dan CheckingAccount

Menerjemahkan diagram ini ke kode Java mengikuti aturan yang sudah dibahas di kelas konsep: panah berujung segitiga kosong dari `SavingsAccount` ke `Account` menjadi `class SavingsAccount extends Account`, baris atribut `-interestRate : double` menjadi field `private double interestRate;`, dan baris method `+printAccountType() : void` menjadi method `public void printAccountType() { ... }`. Method `deposit()`, `withdraw()`, dan `printInfo()` sengaja tidak muncul lagi di kotak `SavingsAccount`, sebab ketiganya sudah diwarisi apa adanya dari `Account` dan tidak perlu dideklarasikan ulang. Mulai dari `SavingsAccount`:

```java
package id.ac.polinema;

public class SavingsAccount extends Account {
    private double interestRate;

    public SavingsAccount(String accountNumber, Customer owner, double balance, double interestRate) {
        super(accountNumber, owner, balance);
        this.interestRate = interestRate;
    }

    public double getInterestRate() {
        return interestRate;
    }

    public void printAccountType() {
        System.out.println("Account type: Savings, interest rate: " + interestRate);
    }
}
```

Figure 2: SavingsAccount.java

Constructor `SavingsAccount` memanggil `super(accountNumber, owner, balance)` di baris pertama untuk membangun bagian yang diwarisi dari `Account`, baru kemudian mengisi `interestRate` miliknya sendiri. Kode ini juga menambahkan `getInterestRate()`, method yang tidak tergambar di diagram: atribut `private` butuh cara dibaca dari luar kelas (kebiasaan encapsulation dari Pertemuan 3), sehingga getter seperti ini wajar ditambahkan meski tidak selalu digambar eksplisit pada setiap diagram. Perbarui `Main.java` untuk mengujinya:

```java
package id.ac.polinema;

public class Main {
    public static void main(String[] args) {
        Customer customer1 = new Customer("Nadia", "0812-0000-0001");
        SavingsAccount acc1 = new SavingsAccount("A001", customer1, 500000, 0.01);
        acc1.withdraw(150000);
        acc1.printInfo();
        acc1.printAccountType();
    }
}
```

Figure 3: Main.java membuat SavingsAccount dan memanggil method warisan maupun method barunya

> ✅ **Checkpoint:** program mencetak `A001 - Nadia - balance: 350000.0` (dari `printInfo()`, method warisan yang tidak ditulis ulang) diikuti `Account type: Savings, interest rate: 0.01` (dari `printAccountType()`, method baru milik `SavingsAccount`).

> ⚠ **Jika gagal:** apabila muncul error `constructor Account in class Account cannot be applied to given types`, periksa apakah `super(...)` di `SavingsAccount` dipanggil dengan urutan dan jumlah argumen yang persis sama dengan constructor `Account` yang dituju.

Langkah 2: CheckingAccount, Subclass Kedua

Tambahkan kelas `CheckingAccount`, subclass kedua dari `Account`, dengan pola penerjemahan diagram yang sama seperti `SavingsAccount`:

```java
package id.ac.polinema;

public class CheckingAccount extends Account {
    private double overdraftLimit;

    public CheckingAccount(String accountNumber, Customer owner, double balance, double overdraftLimit) {
        super(accountNumber, owner, balance);
        this.overdraftLimit = overdraftLimit;
    }

    public double getOverdraftLimit() {
        return overdraftLimit;
    }

    public void printAccountType() {
        System.out.println("Account type: Checking, overdraft limit: " + overdraftLimit);
    }
}
```

Figure 4: CheckingAccount.java

Perbarui `Main.java` untuk menguji kedua jenis rekening:

> ✅ **Checkpoint:** program mencetak empat baris: dua baris pertama untuk `acc1` sama seperti Langkah 1, diikuti `A002 - Sari - balance: 200000.0` dan `Account type: Checking, overdraft limit: 50000.0`. Saldo `acc2` tetap 200000.0 karena penarikan 230000 ditolak: `withdraw()` yang diwarisi dari `Account` hanya mengizinkan penarikan sebesar saldo yang tersedia, dan belum tahu cara memakai `overdraftLimit`. Ini bukan error, melainkan hal

```java
package id.ac.polinema;

public class Main {
    public static void main(String[] args) {
        Customer customer1 = new Customer("Nadia", "0812-0000-0001");
        SavingsAccount acc1 = new SavingsAccount("A001", customer1, 500000, 0.01);
        acc1.withdraw(150000);
        acc1.printInfo();
        acc1.printAccountType();

        Customer customer2 = new Customer("Sari", "0812-0000-0002");
        CheckingAccount acc2 = new CheckingAccount("A002", customer2, 200000, 50000);
        acc2.withdraw(230000);
        acc2.printInfo();
        acc2.printAccountType();
    }
}
```

Figure 5: Main.java menguji SavingsAccount dan CheckingAccount

yang sengaja diamati: atribut baru saja tidak cukup, subclass juga butuh cara menulis ulang perilaku yang diwarisi, dan itulah yang diselesaikan lewat overriding pada Pertemuan 7.

⚠ **Jika gagal:** apabila `overdraftLimit` tidak pernah tersimpan dengan benar, periksa apakah nama parameter constructor tidak keliru tertukar urutannya dengan `balance`.

Langkah 3: Bank Mini Mengelola Beberapa Jenis Rekening

Tanpa mengubah satu baris pun kode Bank, kelas ini sudah bisa menyimpan SavingsAccount maupun CheckingAccount sekaligus, karena keduanya tetap berjenis Account (diwariskan lewat extends). Perbarui `Main.java` untuk membuktikannya:

```java
package id.ac.polinema;

public class Main {
    public static void main(String[] args) {
        Customer customer1 = new Customer("Nadia", "0812-0000-0001");
        SavingsAccount acc1 = new SavingsAccount("A001", customer1, 500000, 0.01);
        acc1.withdraw(150000);

        Customer customer2 = new Customer("Sari", "0812-0000-0002");
        CheckingAccount acc2 = new CheckingAccount("A002", customer2, 200000, 50000);
        acc2.withdraw(230000);

        Bank bank = new Bank(10);
        bank.addAccount(acc1);
        bank.addAccount(acc2);
        bank.printAllAccounts();
    }
}
```

Figure 6: Main.java menaruh SavingsAccount dan CheckingAccount ke dalam Bank yang sama

✅ **Checkpoint:** `printAllAccounts()` mencetak `A001 - Nadia - balance: 350000.0` diikuti `A002 - Sari - balance: 200000.0`, kedua baris memakai bentuk yang sama persis karena `printInfo()` belum ditulis ulang oleh subclass mana pun. Bank belum bisa menampilkan bunga atau limit overdraft masing-masing jenis rekening lewat `printAllAccounts()`.

⚠ **Jika gagal:** apabila muncul error `incompatible types` saat `addAccount(acc1)` dipanggil, periksa apakah parameter `addAccount` di Bank bertipe `Account`, bukan tipe subclass tertentu.

D. Tugas dan Hasil Kerja

Kumpulkan hal berikut sesuai format yang diminta Dosen:

- Screenshot output program setelah Langkah 3.
- **Tugas mandiri:**
  1. Bank Mini menambah satu jenis rekening lagi untuk nasabah korporat. Tambahkan kelas `BusinessAccount extends Account`, dengan atribut `monthlyTransactionFee` dan method baru `printAccountType()`, mengikuti pola yang sama seperti SavingsAccount dan CheckingAccount:
     Buktikan dengan membuat satu `BusinessAccount`, memasukkannya ke Bank bersama rekening lain, lalu memanggil `printAllAccounts()` dan `printAccountType()` miliknya.

```java
package id.ac.polinema;

public class BusinessAccount extends Account {
    private double monthlyTransactionFee;

    public BusinessAccount(String accountNumber, Customer owner, double balance, double monthlyTransactionFee) {
        super(accountNumber, owner, balance);
        this.monthlyTransactionFee = monthlyTransactionFee;
    }

    public double getMonthlyTransactionFee() {
        return monthlyTransactionFee;
    }

    public void printAccountType() {
        System.out.println("Account type: Business, monthly fee: " + monthlyTransactionFee);
    }
}
```

Figure 7: BusinessAccount.java

2. Jawab secara singkat (2-3 kalimat): `Bank.printAllAccounts()` belum bisa menampilkan info khusus tiap jenis rekening (bunga, limit overdraft, atau biaya bulanan), padahal method `printAccountType()` sudah ada di setiap subclass. Mengapa demikian, dan apa yang menurutmu perlu diubah agar `printAllAccounts()` bisa menampilkannya secara otomatis?
3. Jawab secara singkat (2-3 kalimat): apakah `SavingsAccount` sebaiknya dipecah lagi menjadi subclass (misalnya jenis dengan bunga tetap dan jenis dengan bunga berjenjang)? Beri satu contoh subclass yang menurutmu masuk akal beserta atribut barunya, atau jelaskan mengapa pemecahan lebih lanjut belum diperlukan untuk Bank Mini.

E. Kriteria Penilaian

Komponen	Bobot	Kriteria Lengkap (100%)	Kriteria Minimum
Langkah kerja tuntas	40%	Seluruh langkah dijalankan dan berfungsi	Sebagian besar langkah selesai, hasil akhir berjalan
Checkpoint terverifikasi	35%	Semua checkpoint tercapai dan dibuktikan (screenshot/output)	Sebagian checkpoint terbukti
Tugas mandiri	25%	`BusinessAccount` benar dan jawaban konsep tepat	`BusinessAccount` ada meski jawaban belum lengkap